

Technical Assessment - Task 1

DOCUMENTATION

Improvements made to the original code

- ➔ **Video Input Handling** Changed from live camera feed (`cv2.VideoCapture()`) to video file input and added a `video_path` variable at the top for easy configuration. This allows testing with the provided video files.
- ➔ **Removed Age/Gender Detection.** The original code included Caffe models for age and gender detection. Since this wasn't needed for the tracking task and added processing overhead, I removed all the Caffe model loading and detection logic. This simplified the code and improved performance.
- ➔ **Removed Interactive Controls.** The original implementation had keyboard shortcuts (o, p, f, a, space) for toggling different detection modes. Since we're processing videos end-to-end rather than live monitoring, these interactive controls weren't needed. Removed them to simplify the code.
- ➔ **Eliminated Frame Skipping** Original code used `frame_skip = 3` to reduce processing load. However, this causes missed detections and gaps in tracking continuity. Removed frame skipping to ensure every frame is processed for accurate tracking.
- ➔ **Switched to YOLOv8 Built-in Tracking.** This was the main improvement. The original code implemented custom person tracking by comparing bounding box centers between frames.

```
results = model.track(frame, persist=True, tracker="botsort.yaml", conf=0.25, iou=0.5)
```

This leverages YOLOv8's integrated BoT-SORT tracker, which provides much more stable ID persistence.

- ➔ **Code Restructuring:** Split functionality into two main functions: `track_and_export()` handles the tracking pipeline, and `export_clip()` generates clips for specific person IDs. Moved all configuration parameters to the top as constants. This makes the code more maintainable and easier to modify.

TRACKING STRATEGY

Choice of BoT-SORT Tracker YOLOv8 supports multiple tracking algorithms. I selected BoT-SORT because it handles occlusions well by combining motion prediction with appearance-based matching. When a person temporarily disappears behind an object or another person, the tracker can maintain their ID using predicted trajectory and visual features.

```
results = model.track(frame, persist=True, verbose=False, tracker="botsort.yaml", conf=0.25, iou=0.5)
```

The `persist=True` parameter maintains tracking state across frames, preventing ID resets between consecutive frames.

Tracking Mechanism The tracking process works as follows:

1. YOLO detects all persons in the current frame
2. Tracker extracts appearance features and position information
3. The Kalman filter predicts expected positions in the next frame
4. New detections are associated with existing tracks based on feature similarity and predicted positions
5. Successfully matched detections retain their IDs; unmatched detections receive new IDs

CHALLENGES

ID Switching: Even with BoT-SORT, when two people pass very close to each other or cross paths or displacement between frames is very large, the tracker sometimes reassigns IDs. This happens because the appearance features become similar at close proximity. I increased the IOU threshold to 0.5 to make matching more conservative, which reduced this issue

Fast Motion: This was a major issue with ByteTrack - rapid movements caused frequent ID changes. BoT-SORT handles this better but can still lose tracks if the displacement between frames is too large. The motion prediction model has limits on how much movement it can reliably track.

PERFORMANCE CONSIDERATIONS FOR UAV DEPLOYMENT

Current Performance Baseline Using the medium YOLO model (yolov8m.pt) on a laptop, the test video (1280x720, 29 FPS, 1131 frames) was processed in 515.1 seconds, achieving 2.2 FPS. While this is acceptable for offline processing, it's far too slow for real-time UAV deployment.

STEPS TO CHOOSE A TARGET PERSON ID FOR CLIPPING

Steps to Select and Export a Clip:

1. Run the script - it processes the video and generates tracking.csv
2. Analyze which person IDs appear most frequently:
3. Choose an ID that appears in many frames (>50) with good continuity
4. Enter the ID when prompted:
5. The script generates person3_clip.mp4 with only that person's frames

Watch the clip to verify the tracking quality. If there's ID switching or incorrect tracking, try a different ID.

The script produces three output files:

1. tracked_output.mp4 - Complete video with bounding boxes and ID labels overlaid on each detected person
2. tracking.csv - Detection log containing frame numbers, person IDs, confidence scores, and bounding box coordinates
3. personX_clip.mp4 - Individual clip for the selected person ID